AI Assignment 02: TSP-Problem

資工二乙 張惟喆

學號：412262371

1. Introduction

這個報告裡面包含了三種解決 TSP Problem 的演算法：Hill Climb、Simulated Annealing、Genetic Algorithm。而針對每一種演算法都有相對應的改進方法，透過比較這些方法來了解不同演算法的特性，接下來會分成 2. Algorithm Desing、3. Different Algorithm Comparison、4. Parameter Sensitivity Analysis 和 5. Conclusion 幾個部分來探討。

因為這次要實做很多個演算法，並且每個演算法都要跑很多次。在了解的過程中我發現許多演算法都有參數調整的空間，所以我打算把一整個演算法用 Class 包起來，並且整理需要調整的參數，放在方便調整的位置，這樣更適合做研究和分析。

2. Algorithm Design

2.1 Hill Climb (Plain)

Hill Climb 是一種 Local Search 的方法，容易陷入 local optima

1. 每次都會先用隨機的方式產生一條路徑。
2. 再透過隨機交換不同城市，並計算交換完的 `cost`，如果交換完發現有更好的解，就把這條路徑更新到 `best_cost` 和 `best_tour` 中。
3. 總共會隨機交換 `1000 * n` 次。

`Operators and parameters for HC/HC+:`

Operators	**Parameter**
Representation	Permutation of cities (Tour)
Initialization	Random tour
Neighbor	Random Swap two cities
Accept Good Neighbor	Greedy (only accept better neighbor)
Restart Strategy	Fixed amount of restarts (only for HC+)
Stopping Condition	Reach max evaluations 1000 * n

2.2 Hill Climb (Improved)

- 改進的 Hill Climb 使用相同的參數。
- 執行 n 次（例如 n = 5），並且儲存最好的解作為輸出，希望可以藉此逃離 local optima

2.3 Simulated Annealing (Plain)

Simulated Annealing 是一種較好的 Local Search 方法

1. 每次都會先用隨機的方式產生一條路徑
2. 隨機交換不同城市，如果發現更好的 cost 就立即接受新的路徑
3. 如果交換完 cost 反而變大，有一定的機率會接受這條「壞的」路徑，目的是逃離 local optima。根據以下的公式：

$$P = e^{-\Delta/T}$$

(Δ = neighbor_cost − best_cost)

4. 逐步減小 T(T *= alpha)，讓程式越來越不容易接受壞的路徑，這時期的 Simulated Annealing 會表現的更加像個 Hill Climb
5. 總共會隨機交換 1000 * n 次。

Operators and parameters for SA/SA+:

Operators	**Parameters**
Representation	Permutation of cities (Tour)
Initialization	Random tour(SA) / Greedy nearest-neighbor tour(SA+)
Generate Neighbor	Random swap two cities
Accept Good Neighbor	Greedy + little chance to accept bad tour
Cooling Schedule	Follow the formula: T *= alpha
Stop Condition	Reach max evaluations 1000 * n

2.4 Simulated Annealing (Improved)

改進的 Simulated Annealing 參數相同。但是使用 Greedy 來產生第一條路徑，下一個拜訪的城市是尚未拜訪的城市中距離現在城市最近的。以此來讓演算法在最初就從一個還算不錯的解開始優化

2.5 Genetic Algorithm (Plain)

Genetic Algorithm 是一種 Global Search 的方法（因為有 population），模擬生物間的演化過程，整個族群會慢慢朝向好的解演化。

1. 隨機產生一定數量的路徑作為 population
2. 從 population 中隨機選出 k 條路徑，選擇最好的一條作為 parent，一共選擇兩個 parents
3. 對這兩個 parents 做 crossover：隨機挑選 parent1 的某段路徑，複製到 parent2 相應的位置，模擬染色體的基因互換。crossover 完的路徑就叫做 child
4. 有一定的機率（例如：10%）會交換 child 的隨機兩個城市，模擬基因中的突變
5. 將 child 加入 new_generation，等到產生了足夠多的 child，就會一次性覆蓋 population，作為新的起點
6. 自訂每個 Dimension 的評估次數（因為使用 1000 * n 執行時間會較長）

Operators and parameters for GA:

Component	Choice
Representation	Permutation of cities (Tour)
Initialization	Random population
Selection	k-Tournament selection
Recombination	Order crossover (OX)
Mutation	Swap two cities with probability `mutation_rate`
Survivor Selection	Replace the whole population with new generation
Stop Condition	Reach max evaluations

2.6 Genetic Algorithm (Improved)

- 改進的 Genetic Algorithm 在一開始產生 population 的時候會部分使用 greedy tours（例如 25%），這樣起始的 generation 中就會有一些擁有一定質量的 tour，而不是全是隨機的無用路徑。
- 在每次產生新的 generation 之前，會先把原來的 generation 選 top 5 加入到新的 generation 中，這樣可以避免因為 genetic 的算法而使得我們遺失了不錯的解。

Operators and parameters for GA+:

Component	**Choice**
Representation	Permutation of cities (Tour)
Initialization	25% of greedy. 75% of random.
Selection	k-Tournament selection
Recombination	Order crossover (OX)
Mutation	Swap two cities with probability `mutation_rate`
Survivor Selection	Replace whole population with new generation + Elitism
Elitism	Top “elite_count” will add to new generation
Stop Condition	Reach max evaluations

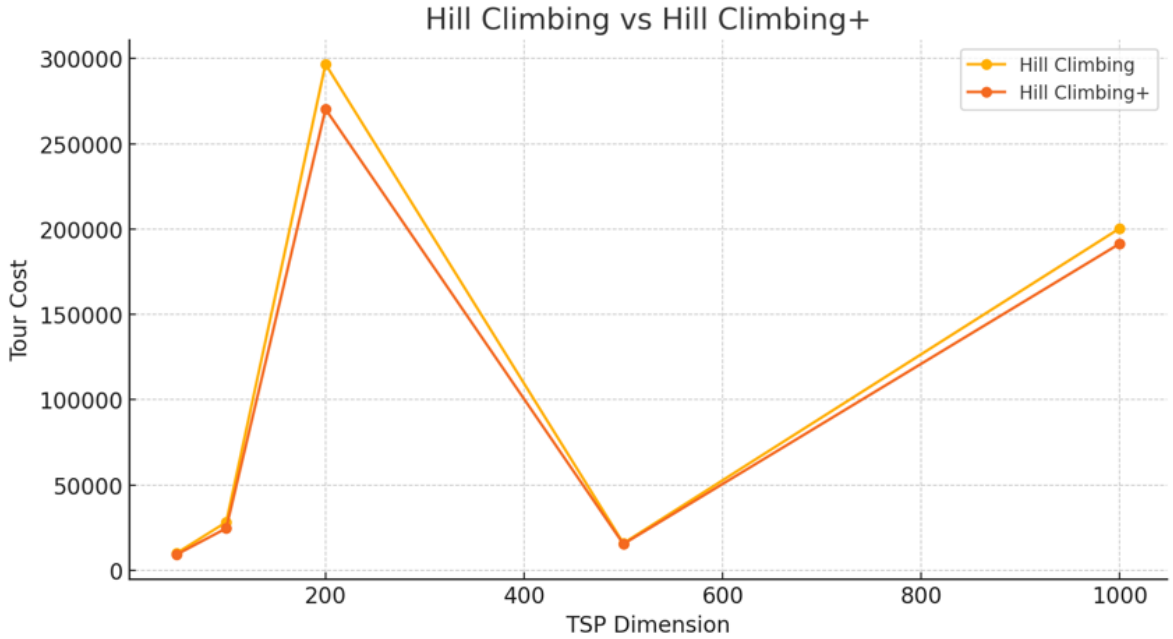
3. Different Algorithm Comparison

3.1 Setup

- Test cases : TSP_Dim = {50, 100, 200, 500, 1000}
- 每個 Test case 執行 10 次
- 測量每個解的 Tour cost, Runtime, Standard deviation

3.2 Hill Climbing vs Hill Climbing+ (Average Tour Length)

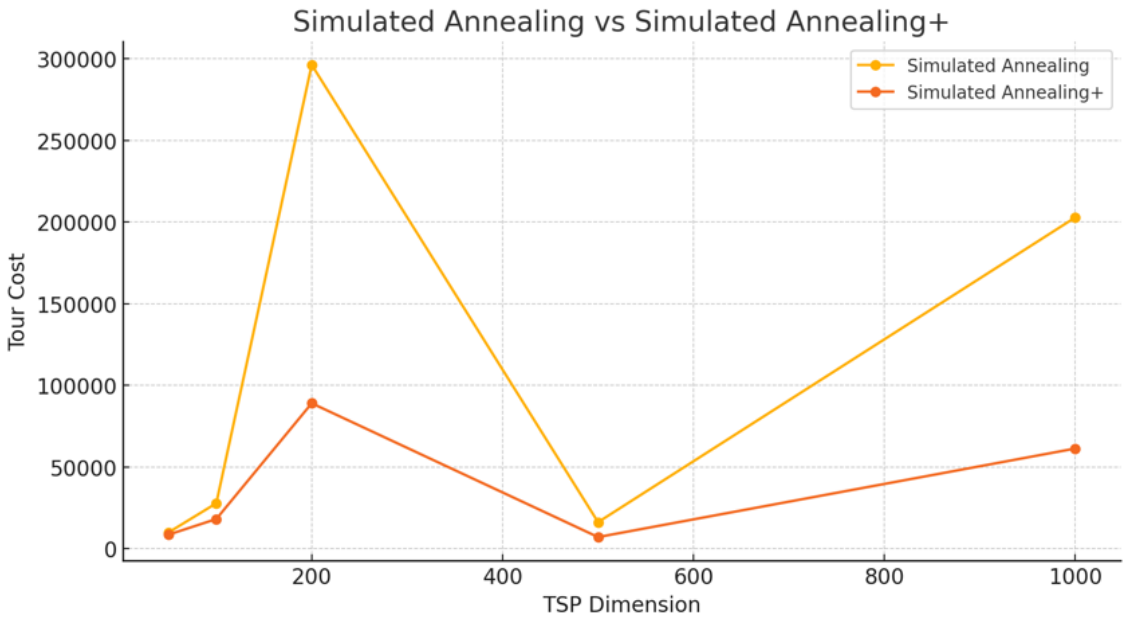
TSP Dim	HC	HC+
50	9995.47	9188.01
100	28162.13	24558.32
200	296752.04	270367.61
500	15795.93	15340.22
1000	200502.81	191509.32



Hill Climb 受限於 local optima，因為他僅接受比當前解更好的解，這種 greedy 的搜尋方法不可能逃離 local optima。所以即使改進後讓他多跑幾次，結果也沒有甚麼顯著的改善，Hill Climb 在 Dimension 大的時候仍然表現出明顯的劣勢。

3.3 Simulated Annealing vs Simulated Annealing+ (Average Tour Length)

TSP Dim	SA	SA+
50	9911.58	8556.93
100	27766.70	18113.48
200	296271.69	89017.23
500	16199.92	7067.54
1000	202849.44	61304.27

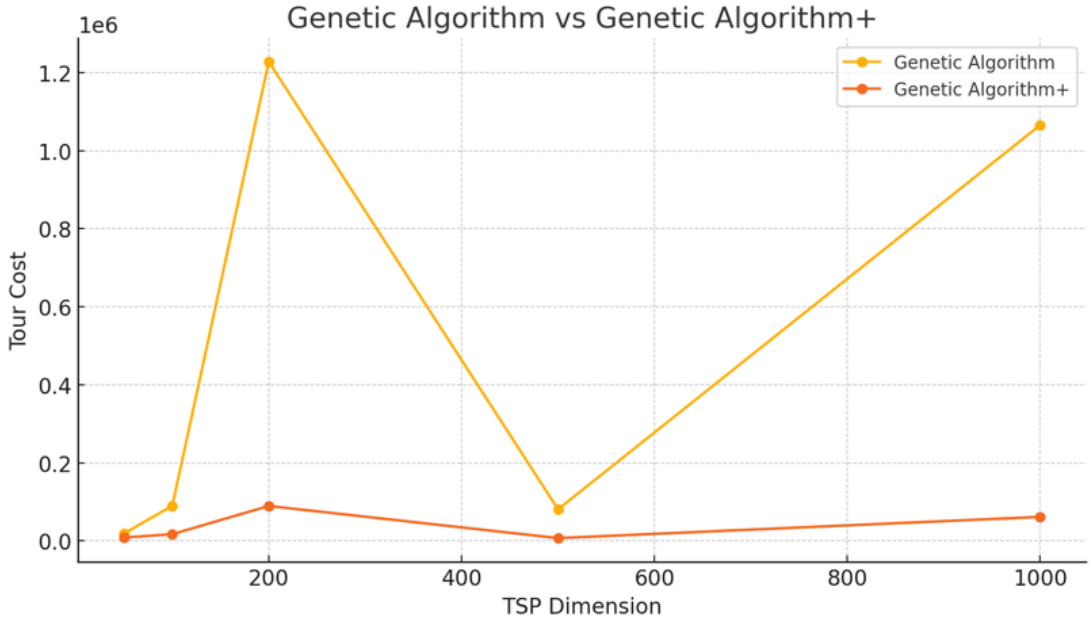


Average Tour Length of SA/SA+

改進版本的 SA+，特別是在 Dimension 較高的時候，可以有效減少三到四倍的 Cost，這表示 Greedy initialization 提供一個很好的起始點，對最終解出的路徑長度有極大的幫助。

3.4 Genetic Algorithm vs Genetic Algorithm+ (Average Tour Length)

TSP Dim	GA	GA+
50	18986.36	8593.33
100	89974.36	17166.77
200	1227817.00	89435.52
500	80966.38	7111.91
1000	1065507.32	61462.73



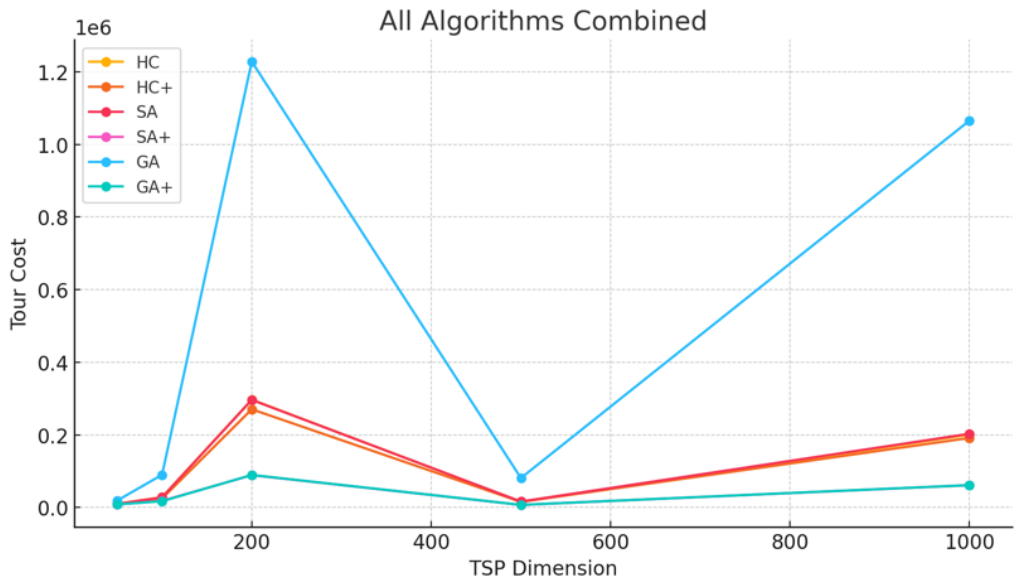
Average Tour Length of SA/SA+

Genetic Algorithm 原本的表現本來非常的差，尤其是在高 Dimension 的時候，總路徑長是其他演算法的數倍。

而 Genetic Algorithm+使用了 Greedy initialization 和 Elite（保留 top 5），有賴於 25%的 greedy tour 和保留 top 5 的機制，使他一下打敗所有演算法，直接成為了橫跨所有演算法中，Average Cost 最好的演算法。這剛好再次印證 Greedy tour 和 Elite 的重要性，幾乎可以主導整個解的好壞程度。

3.5 Combined Comparison of All Algorithms(Cost)

TSP Dim	HC	HC+	SA	SA+	GA	GA+
50	9995.47	9188.01	9911.58	8556.93	18986.36	8593.33
100	28162.13	24558.32	27766.70	18113.48	89974.36	17166.77
200	296752.04	270367.61	296271.69	89017.23	1227817.00	89435.52
500	15795.93	15340.22	16199.92	7067.54	80966.38	7111.91
1000	200502.81	191509.32	202849.44	61304.27	1065507.32	61462.73

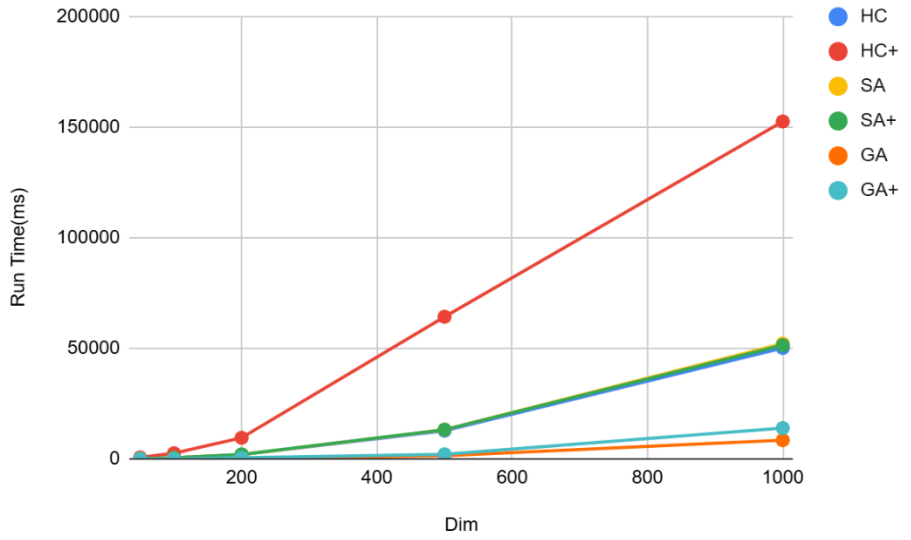


Average Tour Length of all algorithms

從這張圖可以發現，有使用 Greedy Tour 的演算法（Simulate Annealing+(Improve) 和 Genetic Algorithm+(Improve)）在解的長度上都遠超其他演算法，所以可以我們可以知道 Greedy Tour 對演算法的解有很重要的幫助。在這次的實驗中，GA+的表現是最好的，而一些 Local search algorithm(HC, SA)，因為他們有些缺乏探索，有些缺乏把好的解留住的能力，所以 Dimension 上升時無法有效的解決問題。

3.6 Runtime

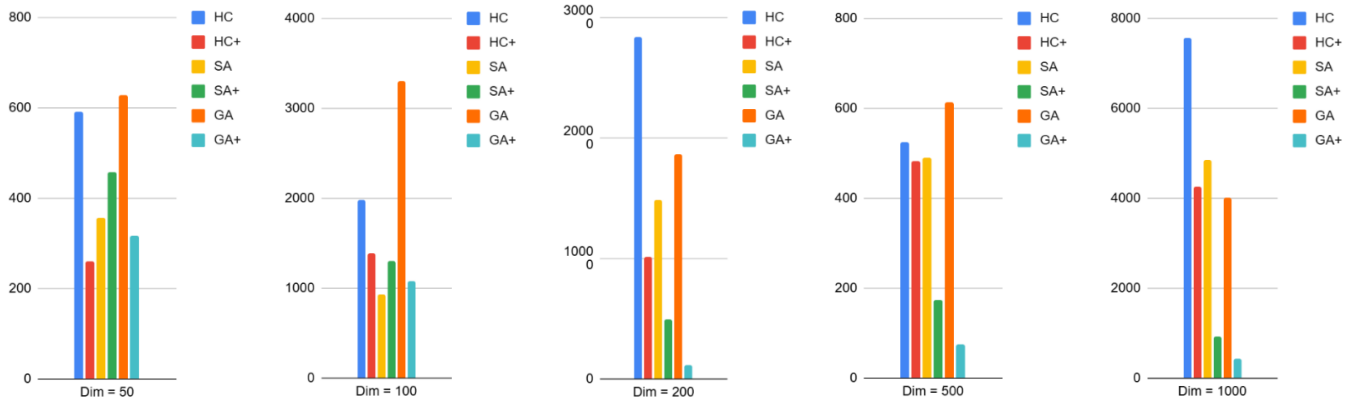
TSP Dim	HC	HC+	SA	SA+	GA	GA+
50	133.80	659.10	140.00	145.80	15.30	17.80
100	495.80	2554.90	516.50	529.40	47.70	54.30
200	1885.70	9540.90	1970.60	2042.30	188.90	255.40
500	12746.30	64299.30	13212.90	13231.90	1433.10	2075.70
1000	50164.70	152633.70	52151.30	51420.70	8472.80	13935.20



Average Runtime of all algorithms

雖然 Genetic Algorithm 我們有手動修改路徑預估的次數，所以執行時間自然是最快的，否則 Genetic Algorithm 大量的計算將會使他榮登執行時間長度榜首。但從這張表仍然可以看出，Simulated Annealing 的算法因為和 Hill Climb 相似，所以執行時間幾乎一樣，而改進版的 Hill Climb 因為只是單純重新跑幾次，所以執行時間呈現線性增長。

3.7 Standard Deviation (Cost)

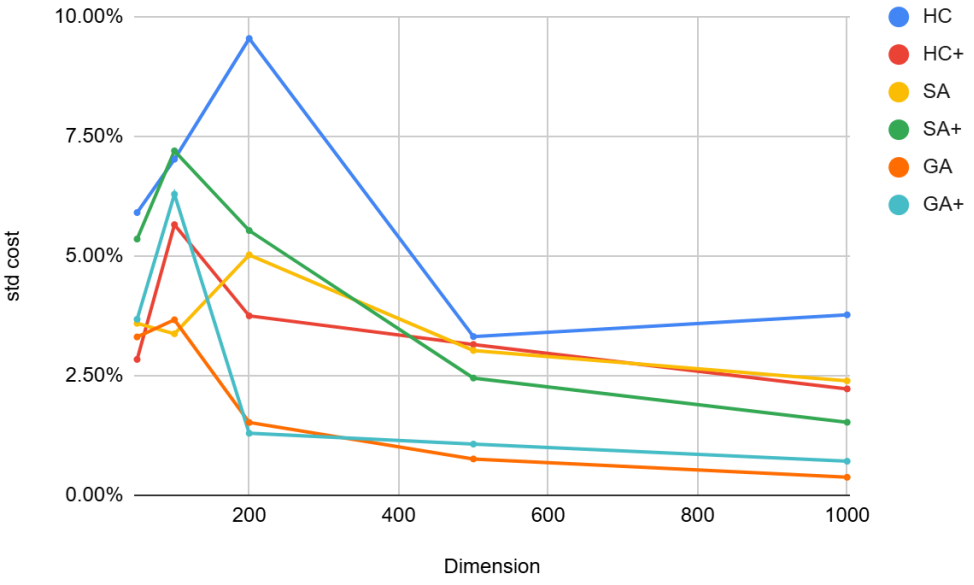


STD Cost of all algorithms in different dimensions

做出這些圖表之後，發現其實很難從這些圖看出甚麼很明顯的訊息，所以我打算把這些標準差轉換成百分比，也就是：

(Std_dev / Avg_cost) * 100%

TSP Dim	HC	HC+	SA	SA+	GA	GA+
50	5.91%	2.84%	3.60%	5.36%	3.31%	3.68%
100	7.03%	5.66%	3.38%	7.20%	3.67%	6.29%
200	9.55%	3.75%	5.03%	5.53%	1.52%	1.30%
500	3.32%	3.15%	3.03%	2.45%	0.76%	1.07%
1000	3.77%	2.22%	2.39%	1.53%	0.38%	0.71%



STD Cost of all algorithms in different dimensions

就可以將這些長條圖整理成折線圖，綜合這兩張圖，可以發現:

- 改進後的演算法基本都有變的更穩定:大多數的結果都維持著:
  - HC+ < HC
  - SA+有時會比 SA 多，或是接近 SA
  - GA+有時比 GA 稍多，但大致上他們都是非常穩定的演算法
- GA 和 GA+在 Dimension 大的時候表現得極其穩定:

  雖然 GA+在 Dim=200 時稍微比 GA 還高，但總體來說當 Dimension 大的時候，GA 和 GA+仍然可保持在 1%左右的變化，是這之中最穩定的演算法。
- HC 很明顯是一個極其不穩定的演算法(特別是在 Dim = 200):

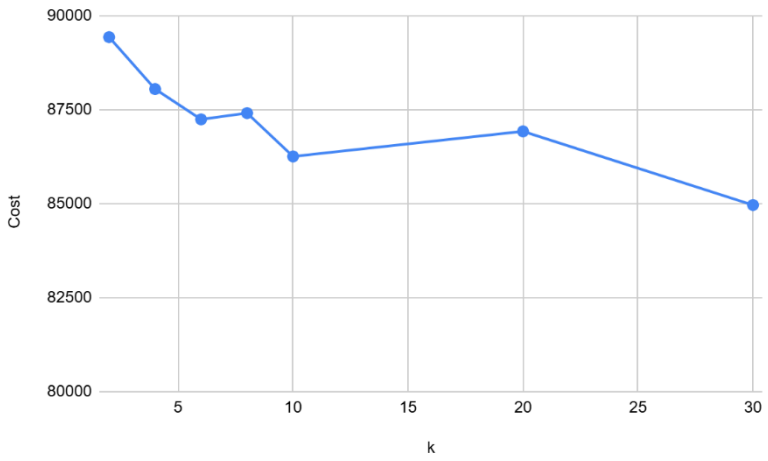
  在小問題的時候，HC 可能可以幸運的猜中解。但特別從 HC 在 Dim=200 時變化來到最高的 9.55%，充分解釋 HC 缺乏廣度的搜尋法特別有可能卡在 local optima，無法找到好的解。但不論在多大的問題，HC 基本都是所有演算法中最不穩定的一種

4. Parameter Sensitivity Analysis (Genetic Algorithm+)

接下來以 Dim = 200 為例子，調整 GA+中的不同參數，固定其他參數，以此來探討不同參數的變化如何影響標準差、Cost 等結果。

- **探討 k 對 Cost 的影響**

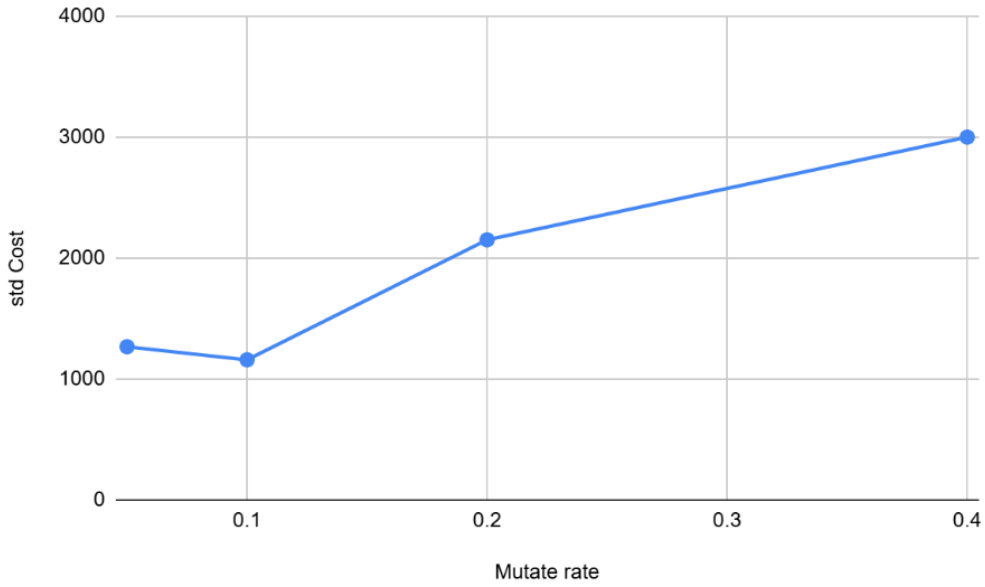
k	Cost	std cost
2	89435.52	1159.34
4	88053.86	1432.14
6	87247.53	1563.6
8	87414.19	1499.16
10	86260.54	1442.74
20	86927.72	1312.45
30	84967.73	1670.34



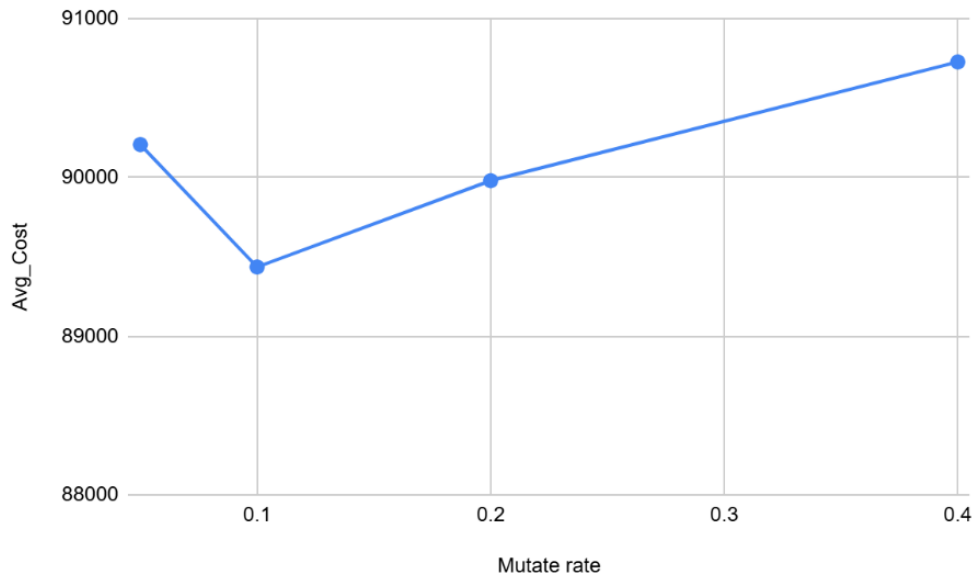
- k 的作用是當我們在 tournament selection 時:
  - 隨機挑選 k 個 population
  - 選擇最好的
- 所以越小的 k，表示挑選越隨機，有點發散的挑選方法可以有機會探索更多路徑，但是缺點是可能遺漏掉當前的 good tour。
- 越大的 k，表示我們越有機會挑中當前的最佳解，所以 old generation 中的 good tour 有很大的機會可以傳承到 new generation 中。
- 實驗中在 k=30 的時候得到最好的解，但同時標準差也是最大的(1670.34)，表示這個結果並不穩定，只是剛好幸運在 k=30 時得到最佳解，所以我認為：
  - K = 2~6(逐漸進步):平均的解逐漸變小，表示選擇的路徑有逐漸向好的解收斂。
  - K = 8~10(平衡點):兼顧了收斂，又不會完全捨棄探索新的路徑(例如 k=10)
  - K = 20~30(過於激進):雖然在這個部分幸運得到的解是最好的，但同時標準差也是最高的，這表示每次的挑選基本都集中在少數幾個個當前的最佳解上，失去了多樣性。所以有些卡在 Local optima 無法逃脫，導致標準差上升。

- **探討 Mutation Rate 對 Standard Deviation(Cost)的影響**

Mutate rate	Cost	std cost	Run Time
0.05	90207.13	1268.03	532.2
0.1	89435.52	1159.34	531.2
0.2	89979.54	2151.87	525
0.4	90727.8	3001.67	531.4



How Mutate rates affect STD cost



How Mutate rates affect average tour length(Cost)

Mutation rate 對解的穩定性和品質有很大的影響，當:

- **Mutation rate = 0.05(太過收斂):**卡在 local optima，沒有探索足夠多的可能，沒辦法找到更好的解

- **Mutation rate = 0.1(平衡點):**這時候的 Avg_Cost 和 Std cost 都是最低的，表示這是非常剛好的點，既保持了足夠的多樣性，也不會死守在 local minimum

- **Mutation rate = 0.2~0.4(過度隨機):**這時候不僅解很糟，連標準差也很大。太大的 Mutation rate 很像 random search，經常破壞 Good tour。這樣的高隨機也使的演算法很不穩定，標準差很大。

- 我還有設計其他值對解影響的實驗，不過效果微乎其微，這裡就只貼出表格，不做更多的探討。

Population Size	Cost	std cost	Run Time
100	89976.76	1881.11	424.6
200	89435.52	1159.34	531.2
400	91060.8	1994.12	718.7
800	88747.24	1930.54	977.9

<Population Size 對 Cost 的影響忽高忽低，初步判斷是因為執行次數只有 10 次，所以效果不顯著>

Max evaluate	Cost	std cost	Run Time
800	89950.38	1465.73	418.1
1200	89435.52	1159.34	531.2
2400	89448.74	2431.02	878
3600	90636.45	2329.07	1222.1

<Max evaluation tour 對 Cost 和 std cost 的結果忽高忽低，初步判斷是因為執行次數只有 10 次，所以效果不顯著>

5. Conclusion

在這份報告中，我們比較了三種解決 TSP 問題的經典的演算法：Hill Climbing, Simulated Annealing, 和 Genetic Algorithm，除了基本的實作，也嘗試改進這些演算法。

透過系統性的測試五種不同的問題大小（從 n = 50 到 n = 1000），每種大小執行 10 次，取

- Average Cost
- Runtime
- 標準差（Solution 的穩定性）

我們可以發現：

- Hill Climbing 只有在問題不大的時候才可以快速得到解，但是基本會卡在 local optima 導致最終解的長度受到限制，基本一開始的初始路徑就直接決定了這次執行中解的品質。
- Simulated Annealing 當使用 greedy initialization 的時候，進步表現顯著，穩定性在 Dimension 大的時候也有進一步的提升。
- Genetic Algorithm，在加入 greedy initialization 和 Elitism 之後，不論是解的穩定性還是解出來的路徑長度都是所有演算法中最小的，即使在 Dimension 非常大的時候(Dim = 1000)，他也可以表現出非常驚人的穩定性和品質，是所有方法中既平衡了時間和解的品質的演算法。
- 在 Parameter sensitivity 的實驗中，調整了(mutation rate, k , etc.)，並且觀察到這些參數如何影響到 Genetic Algorithm 的搜尋過程和最終的結果。